Real-Time Iteration for Aggressive Drone Maneuvers

[redacted]
Department of Electrical and Computer Engineering
The University of British Columbia
[redacted]

***Abstract*—A critical factor for real-time control in fast dynamic environments, such as autonomous drone racing, is the latency between state measurement and action. This is often constrained by the underlying solver's computation time. In this paper, we implement and evaluate a Real Time Iteration (RTI) scheme for Sequential Quadratic Programming (SQP) for a non-linear Model Predictive Controller (NMPC) used to control a drone in a competitive drone racing simulator. We compare the performance of our controller against a SQP NMPC and a PID controller. We show that the RTI scheme successfully reduces the per-step computation time by a factor of 2 while also marginally decreasing the average trajectory tracking RMSE. We investigated the potential trade-offs between sub-optimality and solver latency with the RTI scheme for SQP.**

I. Introduction

The field of autonomous drone racing has emerged as a rigorous testing ground for agile robotic control. Unlike stabilizing a quadrotor at a hover, racing requires operating at the limits of the actuation envelope, where aerodynamic effects and nonlinear dynamics become dominant. The primary challenge lies in minimizing lap time while navigating constrained environments (gates) without collision. This necessitates control strategies that are not only optimal but also robust and capable of improving over time, mimicking the learning process of human pilots. Optimization-based control, specifically Model Predictive Control (MPC), has become the standard for this domain due to its ability to handle constraints explicitly. [1]

Model Predictive Control (MPC) is on track to become one of the most impactful control schemes in the industry. [2] Currently, PID is the most widely used control technology in the industry. One critical advantage PID control holds over MPC is computational speed: every iteration of a PID controller requires a small number of arithmetic operations that can be computed in microseconds. Many control applications, for example spark plug timing in engines, speed controllers for electric motors, and noise canceling headphones, depend on high speed control schemes in order to function properly. However, PID controllers require that every control signal have a desired set point, that the behavior around that set point behaves approximately linearly, and finally that the system operates close to that set point, limiting its effectiveness in cases where the dynamics of the system are highly non-linear. MPC, and Economic-MPC algorithms are much more effective at handling non-linear control schemes, because they determine the next control step at every iteration by solving an optimal control problem over a finite horizon, looking past the first step and further into the future. However, MPC is much more computationally difficult than PID control, crippling its applicability in high speed control applications.

In this paper, we aim to answer the question: how can we achieve the best of both worlds? How can we implement MPC in such a way that it solves non-linear control problems, does so quickly to satisfy the requirements of fast changing dynamics, but without compromising too much on robustness and optimality? To address this, we will implement both an MPC controller that solves a non-linear control problem using SQP, and compare its performance to a high-speed RTI-SQP controller that cuts the computation time of the MPC controller significantly by only partially solving the MPC problem.

The control problem we will investigate is drone flight trajectory tracking. We start with a track which consists of gates that the drone has to fly through in sequence. We then determine a trajectory that the drone should follow along the track, and then aim to make the drone follow the trajectory and complete the track. This problem has non-linear dynamics, and also requires fast control iterations.

II. Preliminaries

Define the system model, control concepts, assumptions, notation, or mathematical background required.

A. Model Predictive Control

Let T be the total time horizon of our problem. Let $\mathbf{x}_t$ be state vectors, $\mathbf{u}_t$ be a control vectors, and $g_t(\mathbf{x}_t, \mathbf{u}_t)$ be some cost function at each point in time $0 \leq t \leq T$, and $g_T(\mathbf{x}_T)$ be the terminal cost function. Let the dynamics of the system be given by some function $f_t(\mathbf{x}_t, \mathbf{u}_t)$. Then, the constrained optimal control problem is given by

$$\begin{aligned} &\min_{u,x} \sum_{t=0}^{T-1} g_t(\mathbf{x}_t, \mathbf{u_t}) + g_T(\mathbf{x}_T) \qquad (1)\\ &\text{subject to: } \mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{x}_0 = \mathbf{X}_0 \\ &\qquad (\mathbf{x}_t, \mathbf{u}_t) \in \mathcal{X}_t \times \mathcal{U}_t \quad \forall t. \end{aligned}$$

Here $\mathcal{X}_t$ and $\mathcal{U}_t$ describe the feasible states and inputs respectively, at time t. This problem solves for an optimal trajectory, and a corresponding sequence of inputs. To implement this as a controller, we solve the above problem at each timestep, by

initializing $\mathbf{x}_0 = \mathbf{X}_{\text{current}}$, apply the first input u_0 as our control signal, and then re-solve the problem with $\mathbf{x}_0 = \mathbf{X}_{\text{next}}$ with a receded time horizon length $T-1 \to T$. In practice, solving this problem at every time step can be infeasible. Model predictive control solves the same problem, but truncates the time horizon, so that instead of optimizing the trajectory over the entire horizon, we optimize over the next N steps, for some $N \leq T$:

$$\begin{aligned} &\min_{u,x} \sum_{t=0}^{N-1} g_t(\mathbf{x}_t, \mathbf{u_t}) + g_N(\mathbf{x}_N) \\ &\text{subject to } \mathbf{x}_{t+1} = f_t(\mathbf{x}_t, \mathbf{u}_t), \mathbf{x}_0 = \mathbf{X}_0 \\ &(\mathbf{x}_t, \mathbf{u}_t) \in \mathcal{X}_t \times \mathcal{U}_t \quad \forall t. \end{aligned}$$

At every step, we solve this problem, initializing $\mathbf{x}_0$ to our initial state $\mathbf{x}$, and applying the first control input $\mathbf{u}_0$ in the sequence of computed inputs.

The optimization problem in Equation (1) is solvable analytically if there are no constraints other than those given by the initial condition and the dynamics, and the cost function is quadratic, with the solution being given by the Riccati equation [3].

B. Nonlinear Trajectory Optimization Problem

In this paper, we are interested in the case where we have non-linear dynamics, and the problem we are looking at has the following properties:

(1) We are trying to minimize the distance between our states and some desired trajectory $\mathbf{x}_{\text{ref}}(t)$, so our cost function is quadratic, with the form

$$g_t(\mathbf{x}_t, \mathbf{u}_t) = \|\mathbf{x}_t - \mathbf{x}_{\text{ref}}\|^2.$$

(2) The dynamics are time invariant, so $f_t = f$ for some function f, but non-linear and potentially very complex.

(3) Constraints: $\mathbf{x}_0 = \mathbf{x}$, and the trajectory $(\mathbf{x}_t)_t$ is propagated by the dynamics function f so that $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$. Also, our inputs are constrained by some lower and upper bounds, that is $\mathbf{u}_{\text{lower}} \leq \mathbf{u}_t \leq \mathbf{u}_{\text{lower}}$.

For the sake of notational convenience, let $\mathbf{x} = (\mathbf{x}_0, ..., \mathbf{x}_N)$ and $\mathbf{u} = (\mathbf{u}_0, ..., \mathbf{u}_{N-1})$, and $\mathbf{y} = (\mathbf{u}, \mathbf{x})$.

Now, for the sake of convenience, denote:

$$\begin{aligned} &f^n(\mathbf{x}_0; \mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{n-1}) \\ &= f(f(\ldots f(f(\mathbf{x}_0, \mathbf{u}_0), \mathbf{u}_1), \ldots), \mathbf{u}_{n-1}). \end{aligned}$$

Then, by substituting

$$\begin{aligned} \mathbf{x}_1 &= f(\mathbf{x}_0, \mathbf{u}_0) \\ \mathbf{x}_2 &= f^2(\mathbf{x}_0, \mathbf{u}_0, \mathbf{u}_1) \\ &\ldots \\ \mathbf{x}_n &= f^n(\mathbf{x}_0; \mathbf{u}_0, \ldots, \mathbf{u}_{n-1}) \end{aligned}$$

into our cost function, we get the following optimization problem:

$$\begin{aligned} &\min_{\mathbf{u}} J(\mathbf{y}) \\ &J(\mathbf{y}) = \sum_{t=1}^{N} \|\mathbf{x}_{\text{ref}}(t) - \mathbf{x}_t\|^2 \\ &\mathbf{x}_0 = \mathbf{x} \\ &\mathbf{x}_t = f^t(\mathbf{x}_0; \mathbf{u}_0, ..., \mathbf{u}_{t-1}), 1 \leq t < N \\ &\mathbf{u}_t \leq \mathbf{u}_{\text{upper}} \\ &-\mathbf{u}_t \leq \mathbf{u}_{\text{lower}}. \end{aligned} \tag{2}$$

Denote the equality constraints $c_i(\mathbf{y}) = 0$ and the inequality constraints $c_j(\mathbf{y}) \leq 0$, with indexing sets $\mathcal{I}$ and $\mathcal{J}$ respectively.

The function J in problem (2) is potentially very complicated because of the dynamics constraint. Even so, this problem must be solved quickly, and at every timestep.

C. PID

Instead of dealing with problem (2), we define the following three error terms:

$$\begin{aligned} \mathbf{e}_p &= \mathbf{x_{ref}} - \mathbf{x} \\ \mathbf{e}_i &\leftarrow \mathbf{e}_i + \mathbf{e}_p \\ \mathbf{e}_d &= \mathbf{v}_{ref} - \mathbf{v}. \end{aligned}$$

The basic intuition behind these three error terms in PID control are that the first term is the difference between our current position and our current desired position, the second term tracks how long we have been away from our desired position to correct "more aggressively" if our previous corrections have not been working well enough, and the third error term corrects for how quickly we are moving away/towards our desired position.

Next, we find some diagonal matrices which correct for these errors, K_p, K_i and K_d, and perhaps some constant term $\mathbf{u}_C$ so that we compute our input as:

$$\mathbf{u} = K_p \cdot \mathbf{e}_p + K_i \cdot \mathbf{e}_i + K_d \cdot \mathbf{e}_d + \mathbf{u}_C$$

By tuning the matrices K_p, K_i, K_d and $\mathbf{u}_C$ such that $\mathbf{u}$ produces a negative feedback situation in our control loop, we can create an effective controller even if the dynamics function f is slightly non-linear [4].

D. Karush-Kuhn-Tucker (KKT) Conditions

If we want to solve problem (2), we need to find the minimum of the cost function J; however, we notice that the problem has constraints. In the unconstrained case, we remember from calculus that if we have a function $f(x)$, a necessary condition for local optimality is $f'(x) = 0$. In solving for this condition, we can find candidates for a local minimum.

The KKT conditions add modifications for the case where we have both inequality and equality constraints, as well as a function we would like to minimize within the feasible region of these constraints. First, consider the inequality constraints,

$c_j(\mathbf{y}) \leq 0$. At any given point $\mathbf{y}$ in our state space, we will have that for each $j \in \mathcal{J}$, either $c_j(\mathbf{y}) = 0$ or $c_j(\mathbf{y}) < 0$. The constraints for which $c_j(\mathbf{y}) = 0$ are called "active". Let $\mathcal{E} \subset \mathcal{J}$ be the active inequality constraints.

Let

$$\mathcal{L}(\mathbf{y}, \lambda) = J(\mathbf{y}) - \sum_{i \in \mathcal{I} \cup \mathcal{E}} \lambda_i c_i(\mathbf{y}).$$

Then, the first-order necessary KKT conditions (that is, the conditions that always apply whenever we are at a local minimum $\mathbf{y}^*$ of the constrained problem, but these conditions don't guarantee that we are at a local minimum) are given by:

(1) Constraints hold: $c_i(\mathbf{y}^*) = 0$, $c_j(\mathbf{y}^*) \leq 0$, $i \in \mathcal{I}$, $j \in \mathcal{J}$.

(2)

$$\begin{aligned} \nabla_{\mathbf{y}} \mathcal{L}(\mathbf{y}^*, \lambda^*) &= 0 \\ \lambda_i^* &\geq 0, \text{ for all } i \in \mathcal{J} \\ \lambda_i^* c_i(\mathbf{y}^*) &= 0, \text{ for all } i \in \mathcal{J} \end{aligned} \tag{3}$$

The way to understand Equation (3) is to see the gradient of J "pushing" against the gradients of the active constraints c_i. When we have optimality, the gradients of the constraints and the function J line up, so that if we were to follow the gradient of J to further decrease its value, we would be violating the constraints: there is no way to move along the constraints that will decrease J. The last equation in the KKT conditions ensures that only the inequality constraints that are active can "push" against J in this manner [5].

E. Sequential Quadratic Programming (SQP)

SQP solves Problem (3) iteratively using quadratic approximations of the cost function and linear approximations of the constraints [6]. Suppose we have a guess, $\mathbf{y}_k$, and corresponding Lagrange multipliers, λ_k, for an optimal solution. Then, we formulate the following problem:

$$\min_p \; J(\mathbf{y}_k) + \nabla J(\mathbf{y}_k)\Delta\mathbf{y} + \frac{1}{2}\Delta\mathbf{y}^\top \nabla^2 \mathcal{L}(\mathbf{y}_k, \lambda_k)\Delta\mathbf{y} \tag{4}$$

$$\nabla c_i(\mathbf{y}_k)\Delta\mathbf{y} + c_i(\mathbf{y}_k) = 0, i \in \mathcal{E} \cup \mathcal{I} \tag{5}$$

Instead of minimizing the quadratic of the cost function, for (4) we use the hessian of the Lagrangian instead of the hessian of the cost function, which warps the space to account for nonlinearities in the constraints. The equations (5) ensure that we move in a direction that obeys the constraints. SQP algorithms iteratively solve for $\mathbf{y}_{k+1} = \mathbf{y}_k + \Delta\mathbf{y}$. Practical algorithms make many different modifications to make computing this problem easier, including simplifying the Hessian, use trust region methods (where the step size is constrained and the algorithm searches for the correct direction to follow), or getting rid of the constraints, and adding them back as violations onto the cost function. See [7] for more background on such techniques, including the method used below.

1) Gauss-Newton Method: In this paper, we will make use of a special approximation for the Hessian: the Gauss-Newton approximation, which applies when our cost function has the form $J(x) = \frac{1}{2}\sum_{j=1}^m w_j r_j(x)^2$, where $r_j : \mathbb{R}^n \to \mathbb{R}$. We can see that our trajectory optimization problem satisfies this formulation. Next, we assume $\nabla^2 \mathcal{L} \approx \nabla^2 J$. Then,

$$\begin{aligned} \nabla J(x) &= \sum_{j=1}^m w_j r_j(x) \nabla r_j(x)^2 \\ \nabla^2 J(x) &= \sum_{j=1}^m w_j (r_j(x)\nabla^2 r_j(x) + \nabla r_j(x) \nabla r_j(x)^\top). \end{aligned}$$

However, if J_r is the Jacobian $J_r(x) = \begin{bmatrix} \nabla r_1(x)^\top \\ \vdots \\ \nabla r_m(x)^\top \end{bmatrix}$, and also, let $W = [w_1, \ldots, w_m]^\top$, so then we can write:

$$\nabla^2 J(x) = J_r W J_r^\top + \sum_{j=1}^m r_j(x) \nabla^2 r_j(x).$$

But then, assuming we are close to the real optimal trajectory, the residuals r_j will be small, so the second term (the sum) will be small, and then we can approximate

$$\nabla^2 J(x) \approx J_r W J_r^\top. \tag{6}$$

III. Problem Formulation

Formally describe the problem. State objectives, task setup, and experimental or theoretical goals.

A. System Model

We applied our controllers to a gymnasium-based drone simulator developed by the Learning Systems Lab at Technical University of Munich [8]. The environment is built on the Mujoco physics simulator, contains several different tracks for the drones to fly through, and has a feature that allows us to record videos of the drone flights. Given a state vector $\mathbf{x}$, we seek to produce a control signal $\mathbf{u}$ at each timestep until the goal is complete. The time steps occur at a frequency of 100 Hz.

The state of the drone is described by the vector $\mathbf{x} = [x, y, z, v_x, v_y, v_z, \theta_r, \theta_p, \theta_y]^\top$. The control signal is given by $\mathbf{u} = [u_r, u_p, u_y, u_T]^\top$, where $[u_r, u_p, u_y]$ are the commands describing the change in the roll, pitch, and yaw of the drone respectively, and u_T is the thrust command.

For the drone being simulated (Craziflie) the roll, pitch, yaw, and thrust of the drone are related to the roll, pitch, yaw, and thrust commands by the following equations:

$$\begin{aligned} T &= 4 \times (\alpha_{ACC}[0] + \alpha_{ACC}[1] u_T) \\ \dot{\theta}_r &= \alpha_R[0]\theta_r + \alpha_R[1] u_r \\ \dot{\theta}_p &= \alpha_P[0]\theta_p + \alpha_P[1] u_p \\ \dot{\theta}_y &= \alpha_Y[0]\theta_y + \alpha_Y[1] u_y \end{aligned} \tag{7}$$

Here, α are parameters that are measured empirically. We can understand u_t to be the thrust command for a single motor, so to get the total thrust, we multiply by 4.

Next, where $R_i(\theta)$ is the rotation matrix about the axis i by angle θ, the direction the drone is "facing" is given by:

$$\begin{aligned}\hat{z} &= R_z(\theta_y)R_y(\theta_p)R_x(\theta_r)\begin{bmatrix}0 & 0 & 1\end{bmatrix}^\top \\ &= \begin{bmatrix}\cos(\theta_r)\sin(\theta_p)\cos(\theta_y)+\sin(\theta_r)\sin(\theta_y)\\ \cos(\theta_r)\sin(\theta_p)\sin(\theta_y)-\sin(\theta_r)\cos(\theta_y)\\ \cos(\theta_r)\cos(\theta_p)\end{bmatrix}\end{aligned}$$

The dynamics of the problem are then described by the following ODE:

$$\dot{v} = T\cdot\hat{z}/m - \begin{bmatrix}0 & 0 & g\end{bmatrix}^\top$$

Here, g is gravitational acceleration, and m is the mass of the drone. Notice that this is not a linear ODE: in the right hand side, $\hat{z}$ is a very complicated nonlinear function of u_r, u_p, and u_y.

To complete our dynamics equations, we have $\frac{d}{dt}(x,y,z) = (v_x, v_y, v_z)$. Write all of these separate dynamics equation in one vector equation with the notation: $\dot{\mathbf{x}} = F(\mathbf{x},\mathbf{u})$ [9].

B. Drone Trajectory

We do not discuss the trajectory planning problem in this paper. Rather, we pre-determine the trajectory by fixing a sequence of waypoints that direct the drone through the course, and fitting a cubic spline to the waypoints. Let T be the time horizon, and f the frequency for the points that we desire for the planned trajectory. We further split up the spline into $T\cdot f$ segments. Then, for $0 \le n < T\cdot f$, we set $\mathbf{x}_{ref}(n)$ to be the starting point of the n-th segment. Our objective is then to minimize the tracking error between this reference trajectory and the actual trajectory:

$$J(\mathbf{x}) = \sum_{n=0}^{T\cdot f-1} \|\mathbf{x}_{ref}(n) - \mathbf{x}_n\|^2$$

1) PID Trajectory Tracking: To compute the error for the PID controller, we compute the desired velocity, $\mathbf{v}_{ref}(n)$ at each segment point along the trajectory if the track is to be completed at the end of the time horizon T. Then, we compute the PID errors with the position, accumulated position (integral), and velocity errors respectively.

2) MPC Trajectory Tracking Cost: We fix N, the number of steps in our finite planning horizon. At each timestep n, we seek to minimize:

$$J(\mathbf{x},\mathbf{u}) = \sum_{k=0}^{N} \|\mathbf{x}_{ref}(n+N) - \mathbf{x}_{n+k}\|^2$$

That is, the cost accumulates the drone's distance from point $\mathbf{x}_{ref}(n+N)$, so the optimal solution will seek to fly to location $\mathbf{x}_{ref}(n+N)$ as quickly as possible, and stay there, within the planning horizon.

C. Constraints

We have the following constraints on the controls:

$$-1 \le u_r, u_p, u_y, u_t \le 1$$

The constants α are tuned so that these limit accurately reflect the constraints on the drones controls.

IV. Main Results

A. PID Implementation

Given the error vectors $\mathbf{e}_p$, $\mathbf{e}_i$, and $\mathbf{e}_d$, we need to determine tuning matrices K_p, K_i, and K_d, so that $\mathbf{u}$ reduces our error. We compute a desired thrust vector, that seeks to minimize the error and accounts for gravity:

$$\mathbf{T}_{des} = K_p\mathbf{e}_p + K_i\mathbf{e}_i + K_d\mathbf{e}_d + 9.81\cdot m_{\text{drone}}\hat{z}$$

From the direction of this vector, we can determine the desired orientation of the drone, θ_{des}, and with equations (7) we can determine the desired commands u_r, u_p, u_y and u_T.

B. SQP NMPC Implementation

In practice, we used the software package Acados [10], which implements a wide range of non-linear optimization algorithms to solve these problems, but we briefly detail their inner workings to help us interpret the results and reason about their speed and application.

1) ODE Integration with RK methods: First, the solver requires a "warm start", a guess, for the optimal trajectory input $\mathbf{u}$, and a starting position x_0. Then, the vector $\mathbf{x}$ can be computed by integrating the dynamics f. We use a fourth-order integration technique for improved accuracy over something like Euler integration, known as the Runge-Kutta [11] method:

$$\begin{aligned}x_{t+1} &= x_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)\\ k_1 &= f(x_t, u_t)\\ k_2 &= f(x_t + k_1\Delta t/2, u_t)\\ k_3 &= f(x_t + k_2\Delta t/2, u_t)\\ k_4 &= f(x_t + k_3\Delta t, u_t)\end{aligned}$$

Then, given a sequence $\mathbf{u}$, we can then quickly and accurately determine the path the drone will follow.

2) Solving the SQP: Our first step in solving the SQP is to simplify (2) [6] [12]. At each computed point $\mathbf{x}_t, \mathbf{u}_t$ we compute the following matrices, which are a linearization of the dynamics function around $(\mathbf{x}_t, \mathbf{u}_t)$.

$$\tilde{\mathbf{x}}_{\mathbf{t+1}} = \Delta t(f(\mathbf{x}_0 : \mathbf{u}_0, \ldots, \mathbf{u}_t) + A_t(\mathbf{x}-\mathbf{x}_t) + B_t(\mathbf{u}-\mathbf{u}_t)),$$

where $\mathbf{x}_t$ and $\mathbf{u}_t$ "warm starts" from the previous control computation, and $A_t = \frac{\partial f}{\partial x}(x_t, u_t)$ and $B_t = \frac{\partial f}{\partial u}(x_t u_t)$ respectively. We are also discretizing the continuous ODE f with Δt. By composing these maps, we can find linear equations for x_t in terms of $\mathbf{u}$ and $\mathbf{x}_0$ at each point in time, so $\mathbf{x}_t = \mathbf{a}'_t(\mathbf{x}_0) + B'_t\mathbf{u}$ for a vector $\mathbf{a}'_t(\mathbf{x}_0)$ and matrix B'_t. We have:

$$J(\mathbf{u}) = \sum_{j=1}^{N} \|B'_t\mathbf{u} - (\mathbf{x}_{ref}(t) - \mathbf{a}'_t(\mathbf{x}_0))\|^2, \tag{8}$$

We then use equation (6) to approximate the Hessian. Acados uses a fast SQP solver called HPIPM, to give us a better guess for the optimal $\mathbf{u}, \mathbf{x}$. We proceed with the following steps:

1) Warm start for $\mathbf{u}$.

2) Integrate to get accurate trajectory $\mathbf{x}$.
3) Linearize the ODE and compose maps. Use the G-N method to get the Hessian and use an SQP solver to get the next iteration of $\mathbf{u}$.
4) Repeat until $\mathbf{u}$ converges to the desired tolerance.
5) Iterate the first step, u_0, in the computed trajectory $\mathbf{u}$.

C. SQP Real-Time Iteration

The Real-Time SQP follows the same steps as the SQP algorithms, with the following modification: only one SQP iteration per control step, where the iteration is warm started with the previous trajectory, and the solution is split apart into a "preparation phase" and a "feedback phase" [12]. The preparation phase is to be started right after the previous control decision has been made, and the computation should complete in time for the next state to be sampled, when the feedback phase is started. The feedback phase is quick, and returns the next control signal shortly after sampling. The algorithm follows the following steps:

1) Preparation Phase
   a) We start with the previous state x_0, the previous control signal $\mathbf{u}$, and use RK integration to compute guesses for $\mathbf{x}$.
   b) We linearize the dynamics to set up the condensed quadratic problem (8) and then the G-N Hessians.
2) Feedback Phase
   a) Having received our updated position when we get our next state sample $\mathbf{x}_0'$, we update the term $a'(\mathbf{x}_0)$ and do a single SQP iteration to get an updated trajectory $\mathbf{u}, \mathbf{x}$, and then we take $\mathbf{u}_0$ to be our next control signal.
   b) This is faster than SQP since we are only doing one iteration, and the linearizations have been computed already. In practice, more computational work is done as well in the preparation phase to reduce the computation time needed in feedback phase.

V. Experimentation

To evaluate the impact of Real-Time Iteration (RTI) on the computational efficiency and control performance of SQP-based NMPC, we conducted three experiments:

- **Experiment 1: Performance in a deterministic setting**: We compared average lap time, tracking RMSE, and solver computation time across all three controllers to establish a performance baseline. This experiment helps us measure the base performance of the controllers in a controlled setting. The results for Experiment 1 are shown in Table I and in Figure 1.
- **Experiment 2: Performance in a stochastic setting**: We introduced additive Gaussian noise to the drone's velocity at varying levels ($\sigma = 0.001, 0.005, 0.01$) to simulate environmental disturbances and sensor noise. A good controller should be robust to such external disturbances. The results for Experiment 2 are shown in Table II

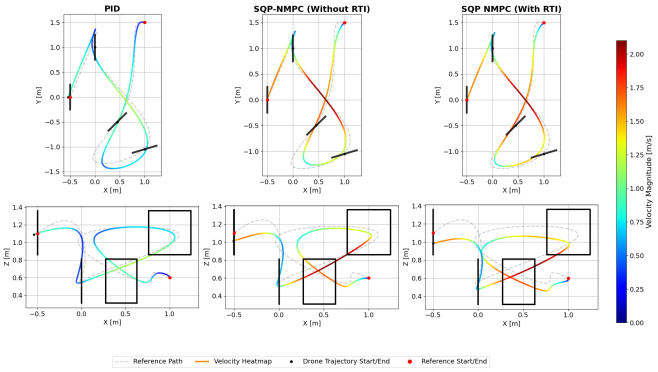


Fig. 1. Drone trajectories using controllers PID, SQP NMPC with and without RTI scheme.

- **Experiment 3: Impact of Planning Horizon**: We analyzed the trade-off between look-ahead capability and solver latency. We hypothesized that for short horizons, the controllers might be myopic and cannot anticipate upcoming maneuvers early enough. It may accelerate aggressively, only to realize too late that it needs to brake or turn which might lead to overshooting, instability, all of which will be reflected by a high RMSE. Conversely, for long horizons, the computational time will increase which means that the controller is more likely to apply "stale" control actions based on outdated information. Therefore, there must be a "sweet spot" for both variants of SQP NMPC controllers. The results for Experiment 3 are shown in Figures 3, 4 and Table III.

VI. Discussion

A. Experiment 1: Performance in a deterministic setting

TABLE I
Controllers Comparison

Solver	Avg Lap Time [s]	Prediction Horizon	Successful Runs	Avg RMSE [m]	Avg Solver Time [ms]
PID Controller	13.80	-	10	0.3065	0.6590
SQP NMPC (Without RTI)	7.08	25	10	0.3134	2.8507
SQP NMPC (With RTI)	7.12	25	10	0.3130	1.3926

1) SQP NMPC maintains a higher velocity profile compared to PID: SQP NMPC with and without the RTI scheme has a faster lap time (averaged across 10 laps) compared to base PID controller. This performance gap is attributed to the differing control strategies. PID reacts conservatively to minimize error which results in slow speeds whereas NMPC optimizes for speed and finds an aggressive "racing line". This difference in strategy is particularly highlighted in Figure 1 where we see the rapid rise in velocity profile in areas between the gates for the SQP NMPC based controllers unlike the PID based controller which maintains a constant velocity profile.

2) RTI scheme reduces SQP NMPC step-wise solver computation time by 51%: This is precisely the result we were aiming for. The standard SQP NMPC is computationally

TABLE II
PERFORMANCE COMPARISON OF CONTROLLERS AGAINST NOISE LEVELS (0.001, 0.005, 0.01) IN THE ACTION SPACE IN THE ENVIRONMENT

	PID			**SQP NMPC (without RTI)**			**SQP NMPC (with RTI)**		
Prediction Horizon	-			25			25		
Metric	**0.001**	**0.005**	**0.01**	**0.001**	**0.005**	**0.01**	**0.001**	**0.005**	**0.01**
Avg Lap Time [s]	13.81	13.8133	13.816	7.09	7.094	7.062	7.116	7.12	7.1289
Successful Runs	10	6	5	10	10	10	10	10	9
Avg RMSE Tracking Error	0.3064	0.3064	0.3066	0.3115	0.3113	0.3123	0.3098	0.3095	0.3126
Avg Solver Time [ms]	0.7588	0.6128	0.6469	2.7204	2.7211	1.6329	1.3461	1.3593	1.2159

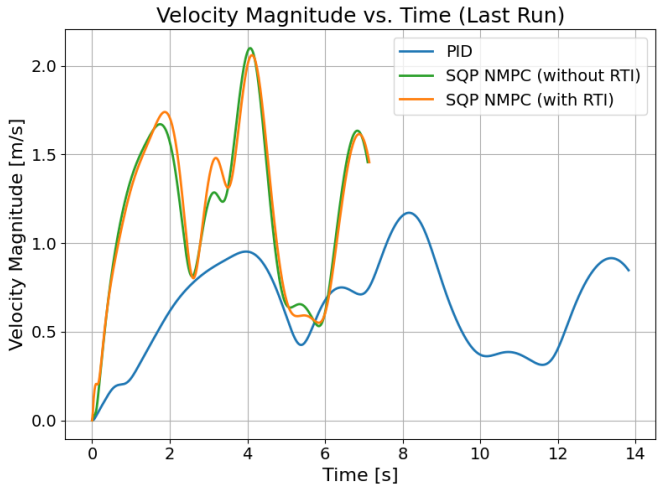


Fig. 2. Velocity profile of drone using controllers PID, SQP NMPC with and without RTI scheme.

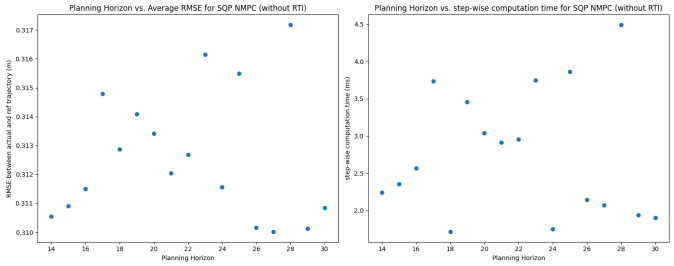


Fig. 3. Average RMSE and step-wise solver computation time vs. increasing planning horizon for SQP NMPC (**without RTI scheme**)

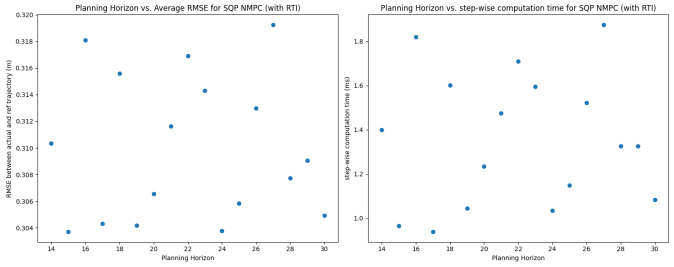


Fig. 4. Average RMSE and step-wise solver computation time vs. increasing planning horizon for SQP NMPC (**with RTI scheme**)

intensive as it performs linearization, quadratization and solves the resulting QP iteratively until it converges to sufficient tolerance every time it receives a state measurement. RTI speeds up this process by splitting it into preparation phase and the feedback phase. An interesting metric to measure is the number of iterations SQP NMPC completes before converging, to see how many iterations we are saving. However, the iterations happen inside Acados' software module and so is difficult to measure.

3) RTI Trade-off: Sub-optimality vs. Latency: The RTI scheme is theoretically sub-optimal because it relies on a linearization based on a slightly outdated (predicted) state and performs only one SQP iteration. This trade-off is reflected in the average lap time where the standard SQP (average lap time of 7.08 seconds) was marginally faster than RTI (7.12s). This difference of 0.04 seconds (approximately 0.5%) reflects the trade-off inherent to the RTI approximation.

B. Experiment 2: Performance in a stochastic setting

Table II displays the controller performance under varying levels of stochasticity in action execution.

1) Robustness to Disturbances: We observed that the PID controller is highly sensitive to disturbances. While its lap times remain consistent when successful, the success rate drops sharply from 100% at noise level 0.001 to 50% at noise level 0.01. This is due to the reactive nature of PID which limits its ability to recover from disturbances during aggressive maneuvers.

Both SQP NMPC variants demonstrate superior robustness mainly due to their ability to anticipate constraints and re-plan the trajectory. The marginal decrease in robustness for the RTI scheme compared to standard SQP at the highest noise level (0.01) might be due to the fact that the RTI scheme uses slightly outdated linearizations.

C. Experiment 3: Impact of planning horizon

It was initially hypothesized that the relationship between planning horizon and RMSE would exhibit a convex 'U-shape' where the RMSE would be high for planning horizon less than the optimal horizon due to "myopia" of the controllers and also high for planning horizon more than the optimal horizon, due to latency issues. It is, however, difficult to identify such a pattern from 3 and 4. However, if we plot the average trajectory tracking RMSE against solver computation time shown in Figure 5, we partially observe the "U" shape that we

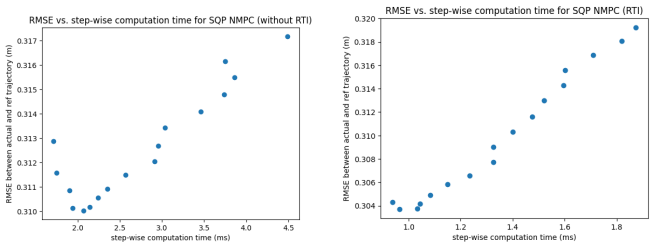


Fig. 5. Avg RMSE vs step-wise computation time for SQP NMPC without RTI scheme (left) and with RTI scheme (right)

previously hypothesized. We notice that there is an optimal point, beyond which increasing the solver computation time leads to a sharp increase in RMSE. The plot also demonstrates that latency is the dominant source of error since the trajectory tracking RMSE scales linearly with latency. In addition to this, the optimal latency with the RTI scheme (smallest RMSE) is roughly 2.1 times less than the optimal latency without the RTI scheme.

TABLE III
EFFECT OF RTI SCHEME ON SQP NMPC PERFORMANCE WITH INCREASING HORIZON

Solver	**Avg RMSE [m]**	**Avg Solver Time [ms]**
SQP NMPC (Without RTI)	0.3126 ± 0.0022	2.312 ± 0.678
SQP NMPC (With RTI)	0.3099 ± 0.0052	1.232 ± 0.271

TABLE IV
OPERATIONAL REGION OF PLANNING HORIZON FOR SQP NMPC VARIANTS

Solver	**Planning Horizon Minimum**	**Planning Horizon Maximum**
SQP NMPC (Without RTI)	14	34
SQP NMPC (With RTI)	11	30

1) Difference in Stability Boundaries: The two RTI schemes have different ideal regions for planning horizons and this is shown in Table IV. Below the minimum planning horizon, the controllers fail to pass all 4 gates and similarly above the maximum planning horizon, the controllers also fail to pass all 4 gates. The operating ranges where the controllers succeed are notably differently and this gives us insight into the critical trade-off between feedback speed (latency) and optimization quality. We notice that the minimum horizon for SQP with RTI is lower than the standard SQP. This is because standard SQP has higher computational latency than RTI (almost double). At lower horizons, the controllers struggle with lack of foresight and this coupled with the delayed feedback is insufficient to stabilize the drone during aggressive maneuvers. For RTI, the low latency provides the necessary rapid feedback to stabilize the drone which compensates for the lack of foresight. Our main takeaway from the lower bound for the planning horizon is that when predictive capability is weak, fast feedback is more critical than optimization accuracy.

On the other end of the spectrum, we notice that RTI fails at shorter maximum horizon than standard SQP. This is paradoxical since, RTI has a faster solver so it should avoid latency issues with increasing planning length. One explanation we could propose from this observation is that the limiting factor at extreme horizons is numerical stability instead of latency:

- As N becomes very large, the optimization problem accumulates small numerical errors for each planning horizon step. This leads to an ill-conditioned optimization problem.
- Since RTI scheme relies on a single SQP iteration and uses approximations to get the Hessian, this single step can be numerically unstable.

While both methods eventually fail beyond a certain planning horizon due to latency, the iterative nature of standard SQP may provide better numerical stability at the extreme maximum horizons.

*2) RTI scheme **improves** average RMSE:* One notable observation is that the RTI scheme *reduces* average trajectory tracking RMSE. This particular observation is highlighted in Table III where we compute the average trajectory tracking RMSE across different planning horizons for SQP with and without RTI. We also observe this behavior in the results of the previous two experiments as well in Table I and Table II.

The SQP NMPC with RTI scheme has slightly lower RMSE compared to SQP NMPC without RTI scheme. This result is initially paradoxical given the fact that RTI scheme is *sub-optimal*. Upon deeper reflection, we think this paradox can be resolved if we understand that there is a critical distinction between mathematical optimality (*solving an optimization problem exactly*) and closed-loop performance (*how well is the reference tracked in real-time*). We had naively assumed that RMSE represented the accuracy of the mathematical optimality of our solver, whereas in reality it measures the latter: closed-loop performance. RTI SQP outshines the standard SQP for average trajectory tracking RMSE (closed loop performance) because of the reduction in computational latency due to the RTI scheme. Suppose the time taken for the standard SQP solver to generate a control input is Δt. Since the solver with the RTI scheme is roughly *twice* as fast, the time taken for the RTI scheme solver to generate a control input is $\sim \frac{\Delta t}{2}$. During a sharp turn, a state measurement x_t is received at time-step t. In standard SQP implementation, the computation starts at time t and by the time the solver returns the perfect control input, the drone has moved to the state $x_{t+\Delta t}$. Therefore, the "optimal" control action generated by the standard SQP is "stale" since the optimal control action was generated for a state that *existed* Δt ms ago. In aggressive maneuvers, the drone's dynamics are so fast that the state changes rapidly. RTI is able to deliver a control input *much sooner*. Therefore, in dynamics systems, **applying a good enough control input quickly** is often better than applying **the perfect control input late**. By reducing latency, RTI

scheme makes the feedback loop tighter which improves the stability of the drone, especially during rapid turns. Indeed if one looks at the velocity profile between the MPC with SQP with and without RTI scheme (Figure 2), one notices that the RTI scheme matches the velocity profile of the standard SQP very closely except the maximum velocity under the RTI scheme is **slightly less** around the turns. This makes all the difference for the lower average RMSE.

VII. Conclusion

Through this project, we demonstrate the implementation and evaluation of SQP NMPC with the Real-Time Iteration (RTI) scheme for aggressive drone maneuvering. We arrived at the following conclusions:

- **NMPC is essential for racing:** Non linear MPC provides dramatic improvements in average lap time (nearly two times faster) and robustness compared to standard PID.
- **RTI scheme reduces computation time by 51% compared to standard SQP NMPC**
- The sub-optimality of RTI resulted in a negligible impact on average lap time ($< 0.5\%$ increase) and only a minor reduction in robustness at the highest disturbance (noise) levels.
- Reduced latency offered by RTI can potentially improve tracking performance (lower RMSE) by enabling faster closed loop feedback stabilization.

Therefore, we conclude that SQP NMPC with the RTI scheme offers the best balance of high-speed performance, computational efficiency, and robustness for the application of drone racing with aggressive maneuvers.

Acknowledgments

We are grateful to Professor Alberto Padoan for teaching us the fundamentals of Model Predictive Control.

References

[1] A. Romero, S. Sun, P. Foehn, and D. Scaramuzza, "Model predictive contouring control for time-optimal quadrotor flight," *IEEE Transactions on Robotics*, vol. 38, pp. 3340–3356, 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID:248512555

[2] T. Samad, M. Bauer, S. Bortoff, S. Di Cairano, L. Fagiano, P. F. Odgaard, R. R. Rhinehart, R. Sánchez-Peña, A. Serbezov, F. Ankersen, P. Goupil, B. Grosman, M. Heertjes, I. Mareels, and R. Sosseh, "Industry engagement with control research: Perspective and messages," *Annual Reviews in Control*, vol. 49, pp. 1–14, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1367578820300080

[3] J. Rawlings, D. Mayne, and M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017. [Online]. Available: https://books.google.ca/books?id=MrJctAEACAAJ

[4] K. J. Åström, "Pid controllers: Theory, design, and tuning," 1995. [Online]. Available: https://api.semanticscholar.org/CorpusID:58689302

[5] S. Boyd and L. Vandenberghe, *Convex Optimization*, ser. Berichte über verteilte messysteme. Cambridge University Press, 2004, no. pt. 1. [Online]. Available: https://books.google.ca/books?id=mYm0bLd3fcoC

[6] J. Nocedal and S. Wright, *Numerical Optimization*, ser. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006. [Online]. Available: https://books.google.ca/books?id=VbHYoSyelFcC

[7] J. E. Dennis and R. B. Schnabel, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*. Society for Industrial and Applied Mathematics, 1996. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611971200

[8] L. Systems and T. U. o. M. Robotics Lab, "Lsy drone racing simulator," 2024, accessed: YYYY-MM-DD. [Online]. Available: https://github.com/utiasDSL/lsy_drone_racing

[9] R. E. Mahony, V. R. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," *IEEE Robotics & Automation Magazine*, vol. 19, pp. 20–32, 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:1081012

[10] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, pp. 1–37, 10 2021.

[11] E. Hairer, S. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, ser. Springer Series in Computational Mathematics. Springer Berlin Heidelberg, 2008. [Online]. Available: https://books.google.ca/books?id=F93u7VcSRyYC

[12] M. Diehl, H. Bock, and J. Schlder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM Journal on Control and Optimization*, vol. 43, pp. 1714–1736, 07 2006.